

MINISTRY OF EDUCATION AND SCIENCE OF UKRAINE

NATIONAL TECHNICAL UNIVERSITY OF UKRAINE

" IHORY SIKORSKY KYIV POLYTECHNIC INSTITUTE"

A. M. Sergiyenko

A. A. Molchanova

CAD of computer systems

Lab Work 3

Elementary function calculation block

Kulubecioglu Mehmet

Class Number 12

IM-14 FIOT

Kyiv

IHORY SIKORSKY KYIV POLYTECHNIC INSTITUTE

2024

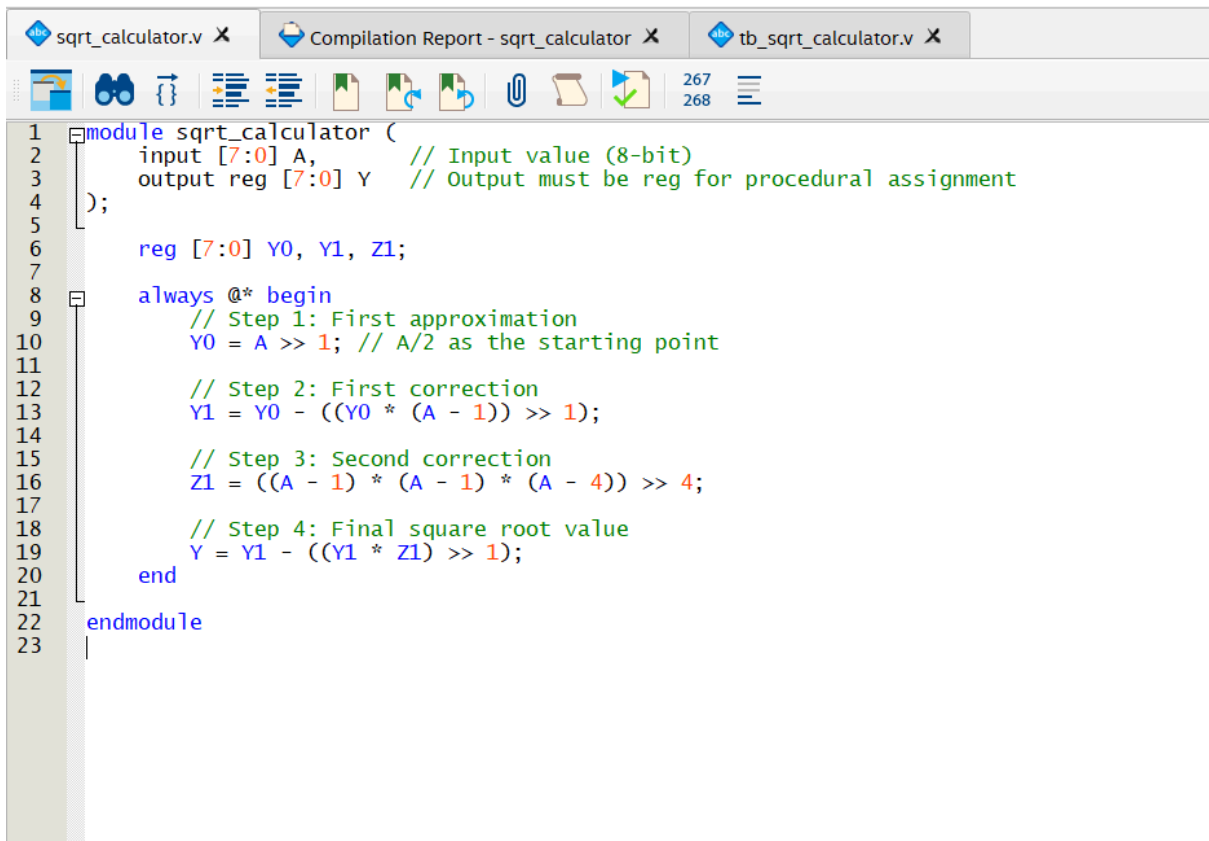
1. Code Explanation (Design File):

This code defines a hardware module that performs square root calculation. The module has two main components:

Input signal: Represents the number for which the square root is to be calculated.

Output signal: Represents the result of the square root calculation.

The code uses combinational logic to apply a simplified operation on the input signal to calculate the square root. This design doesn't include any timing elements; whenever the input value changes, the output is immediately updated.



```
1 module sqrt_calculator (
2     input [7:0] A,          // Input value (8-bit)
3     output reg [7:0] Y      // Output must be reg for procedural assignment
4 );
5
6     reg [7:0] Y0, Y1, Z1;
7
8     always @* begin
9         // Step 1: First approximation
10        Y0 = A >> 1; // A/2 as the starting point
11
12        // Step 2: First correction
13        Y1 = Y0 - ((Y0 * (A - 1)) >> 1);
14
15        // Step 3: Second correction
16        Z1 = ((A - 1) * (A - 1) * (A - 4)) >> 4;
17
18        // Step 4: Final square root value
19        Y = Y1 - ((Y1 * Z1) >> 1);
20    end
21 endmodule
22
23
```

2. Code Explanation (Testbench File):

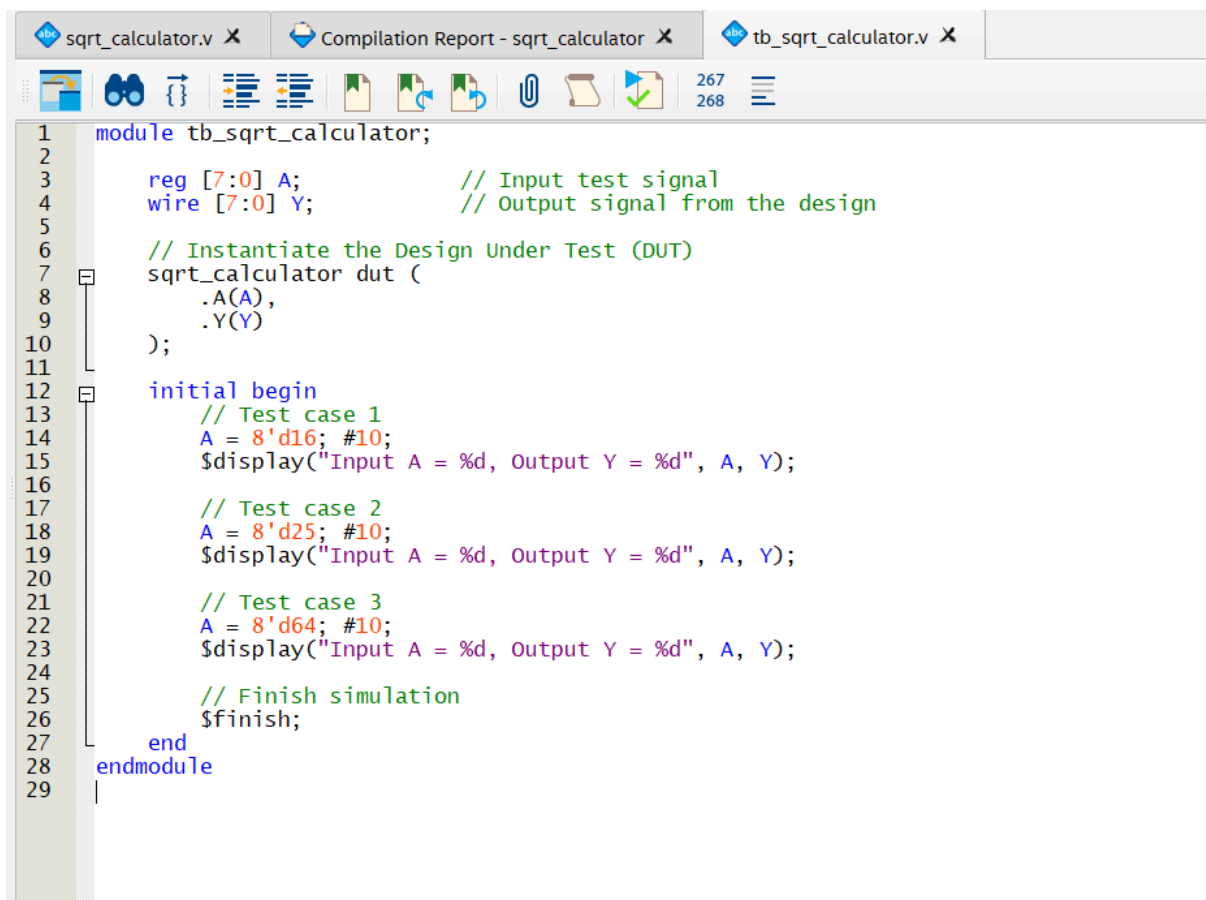
This code is used to test the design file. The testbench applies unit tests to the design module and checks the correctness of the results. The testbench has three main functions:

Generate input signals: It sends different test values to the design module.

Observe the outputs: It checks if the results from the design module are correct.

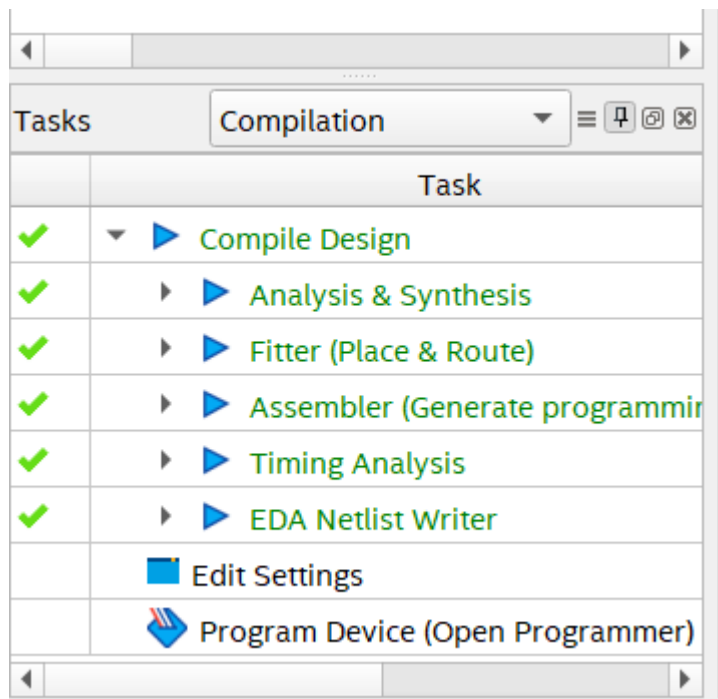
Control the simulation: It manages the timing and flow of the simulation.

The testbench is used to verify if the design produces the expected outputs, ensuring that the design is functioning as intended.

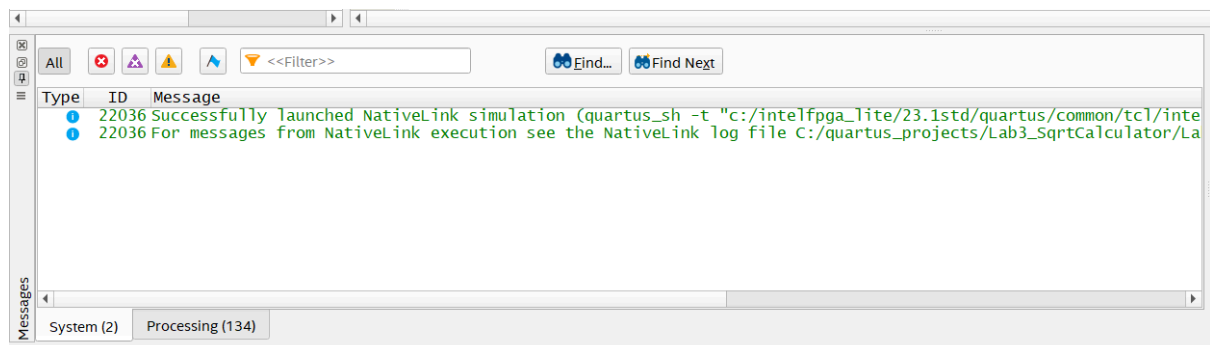


```
1 module tb_sqrt_calculator;
2
3     reg [7:0] A;           // Input test signal
4     wire [7:0] Y;         // Output signal from the design
5
6     // Instantiate the Design Under Test (DUT)
7     sqrt_calculator dut (
8         .A(A),
9         .Y(Y)
10    );
11
12    initial begin
13        // Test case 1
14        A = 8'd16; #10;
15        $display("Input A = %d, Output Y = %d", A, Y);
16
17        // Test case 2
18        A = 8'd25; #10;
19        $display("Input A = %d, Output Y = %d", A, Y);
20
21        // Test case 3
22        A = 8'd64; #10;
23        $display("Input A = %d, Output Y = %d", A, Y);
24
25        // Finish simulation
26        $finish;
27    end
28 endmodule
29
```

3- Compilation details



Tasks		Compilation			
	Task				
✓	▶	▶	▶	Compile Design	
✓	▶	▶	▶	Analysis & Synthesis	
✓	▶	▶	▶	Fitter (Place & Route)	
✓	▶	▶	▶	Assembler (Generate programming file)	
✓	▶	▶	▶	Timing Analysis	
✓	▶	▶	▶	EDA Netlist Writer	
				Edit Settings	
				Program Device (Open Programmer)	



Type	ID	Message
Information	22036	Successfully launched NativeLink simulation (quartus_sh -t "c:/intelfpga_lite/23.1std/quartus/common/tcl/intel...")
Information	22036	For messages from NativeLink execution see the NativeLink log file C:/quartus_projects/Lab3_SqrtCalculator/La...

4- Compilation Report

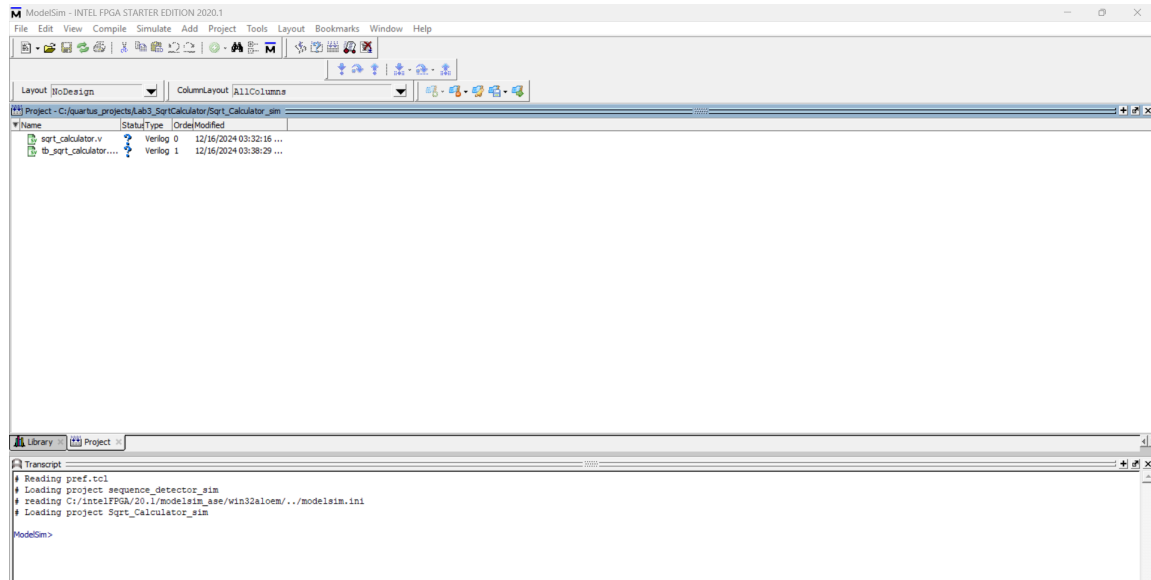
Flow Summary		
Filter		
Global Settings	Flow Status	Successful - Mon Dec 16 15:39:39 2024
	Quartus Prime Version	23.1std.1 Build 993 05/14/2024 SC Lite Edition
Messages	Revision Name	sqrt_calculator
	Top-level Entity Name	sqrt_calculator
	Family	Cyclone V
	Device	5CGXFC7C7F23C8
	Timing Models	Final
	Logic utilization (in ALMs)	18 / 56,480 (< 1 %)
	Total registers	0
	Total pins	16 / 268 (6 %)
	Total virtual pins	0
	Total block memory bits	0 / 7,024,640 (0 %)
	Total DSP Blocks	4 / 156 (3 %)
	Total HSSI RX PCSs	0 / 6 (0 %)
	Total HSSI PMA RX Deserializers	0 / 6 (0 %)
	Total HSSI TX PCSs	0 / 6 (0 %)
	Total HSSI PMA TX Serializers	0 / 6 (0 %)
	Total PLLs	0 / 13 (0 %)
	Total DLLs	0 / 4 (0 %)

5- Modelsim

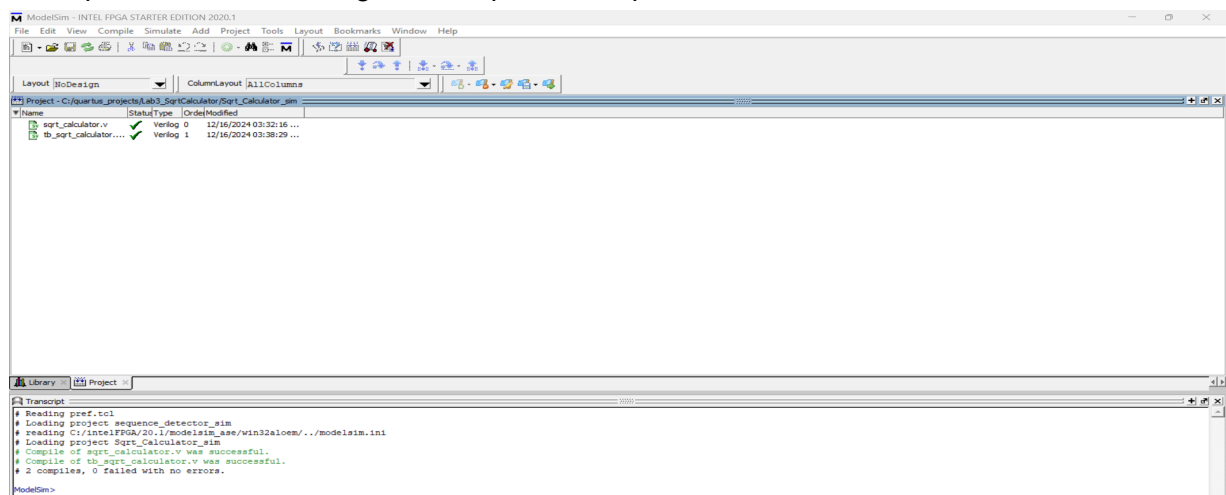
- I created a new project by selecting File > New > Project.

The project name was set to sequence_detector_sim, and the file path was selected.

- I added the design files (such as sequence_detector.vhd and sequence_detector_tb.vhd) to the project using Add > To Project > Existing File.

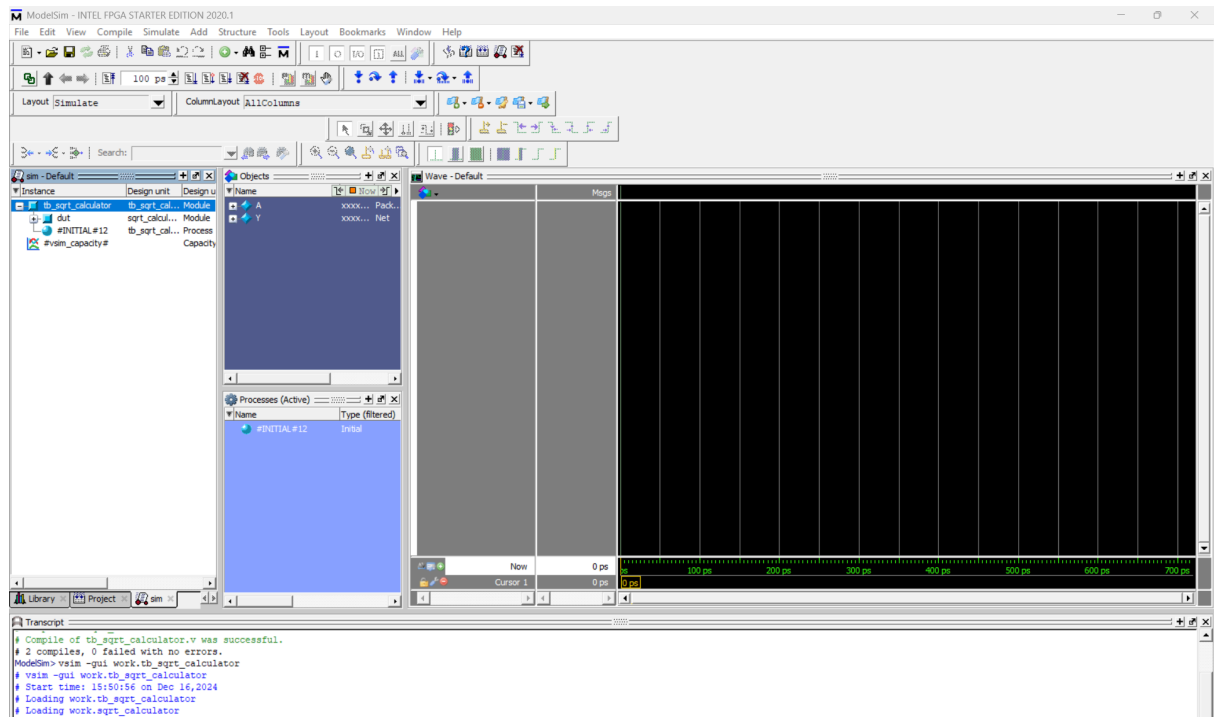


I compiled all the files using the Compile > Compile All command.



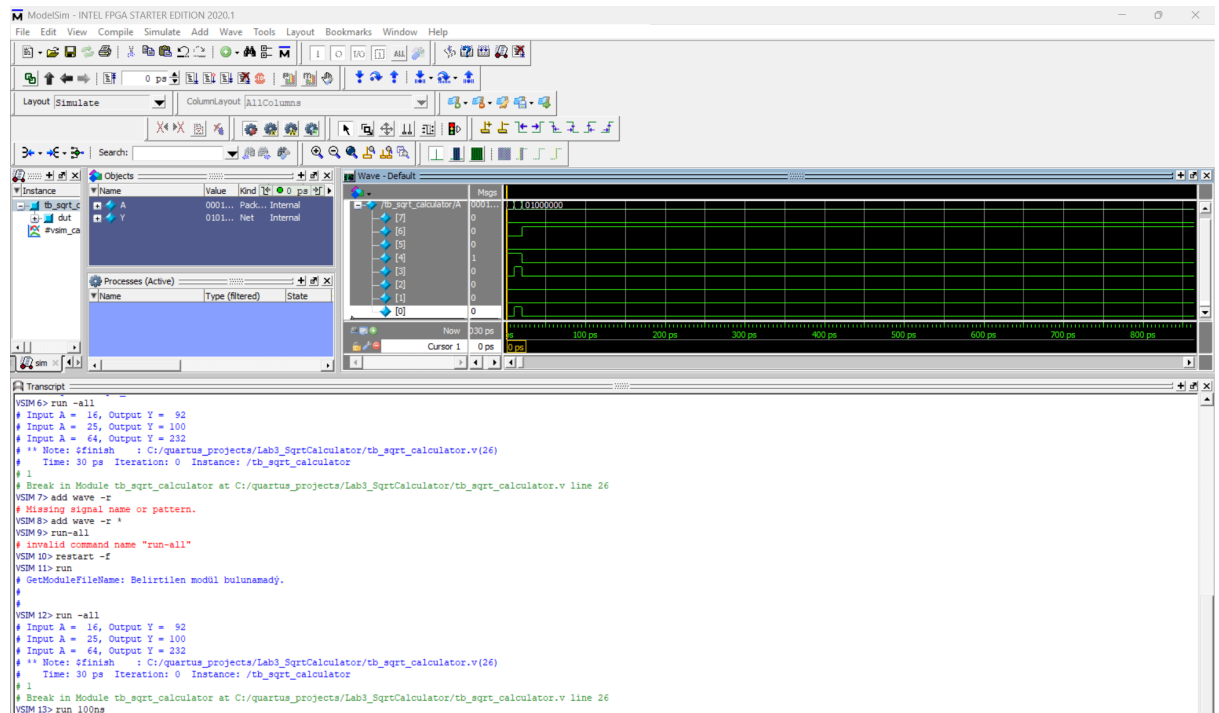
6 - Simulation With Modelsim

- I started the simulation using Simulate > Start Simulation and selected the testbench file.



7- To Wave

- I added all the signals in the project to the waveform by selecting Add > To Wave > All Items in Design.



Control questions

1. Why are there entity and architecture sections in a VHDL project?

In VHDL, the entity and architecture sections are used to define different aspects of a module:

- Entity: The entity describes the interface of the module. It defines the inputs and outputs of the module, essentially specifying the external connections to the module.
- Architecture: The architecture describes the internal behavior of the module. It specifies how the module works, including its internal components, logic, and state transitions. You can have multiple architectures for a single entity, providing different implementations or behaviors.

2. How are blocks with sequential behavior in time described in VHDL?

In VHDL, sequential behavior is described using process blocks. A process defines a set of instructions that are executed in a sequence, triggered by specific events or signals. The process is typically used to describe behavior such as state machines, counters, or any other behavior that depends on the sequential execution of operations.

```

process (clk)
begin
    if rising_edge(clk) then
        -- sequential behavior here
    end if;
end process;

```

3. In what ways can a signal edge event be specified in a process operator?

In VHDL, edge events for signals can be specified in the process operator using the following:

- if rising_edge(signal): This checks for a rising edge (from 0 to 1) of the signal.
- if falling_edge(signal): This checks for a falling edge (from 1 to 0) of the signal.
- if signal'event and signal = '1': This checks for a positive edge manually, detecting when the signal changes to 1.

```

process (clk)
begin
    if rising_edge(clk) then
        -- Sequential behavior on rising edge
    end if;
end process;

```

4. How is testing of modules developed using a VHDL simulator performed?

Testing in a VHDL simulator is typically performed using a **testbench**. A testbench is a piece of VHDL code that instantiates the module (the Unit Under Test, UUT), applies input signals, and observes the output. The following steps are usually involved:

1. **Write testbench code** that defines input signals and applies various test cases.
2. **Instantiate the UUT** in the testbench.
3. **Simulate** the design using the simulator (like ModelSim or Vivado) and observe the output.
4. **Check** if the output matches the expected results.
5. **Debug** any errors by examining the simulation waveform and output.

5. How to generate test signals in a testbench?

Test signals in a VHDL testbench can be generated by defining signal assignments within the testbench process. These assignments provide specific values to the signals at different simulation times.

```
process
begin
    -- Apply test vectors
    A <= "0001"; -- Apply value to signal A
    wait for 10 ns;
    A <= "0010"; -- Change value of A
    wait for 10 ns;
    -- Continue with more test vectors
    wait;
end process;
```

6. How to check the correct operation of a module in a testbench?

To check the correct operation of a module in a testbench, you should:

1. **Compare outputs** with expected results. For example, using assert statements to check if the output is correct.
2. **Use simulation waveforms** to visually inspect the behavior of signals over time.
3. **Include multiple test cases** with varying inputs to verify the functionality under different conditions.
4. **Print the results** to the simulation transcript using \$display or report statements.

```
process
begin
    if (output_signal = expected_value) then
        report "Test passed" severity note;
    else
        report "Test failed" severity error;
    end if;
    wait;
end process;
```

7. What steps does a project go through in FPGA CAD to obtain a firmware file?

In FPGA CAD (Computer-Aided Design), the typical steps to obtain a firmware file (bitstream) are:

1. **Design Entry:** Write the VHDL/Verilog code or create the design using a graphical tool (e.g., block diagram).
2. **Synthesis:** Convert the high-level design (VHDL/Verilog) into a netlist, representing logic gates and components.
3. **Mapping:** Map the logic gates to specific resources available on the FPGA device (LUTs, flip-flops, etc.).
4. **Placement:** Place the mapped logic on the actual FPGA resources, deciding the physical location of logic blocks.
5. **Routing:** Route the connections between the placed logic blocks.
6. **Bitstream Generation:** After placement and routing, the tool generates a configuration file (bitstream) that configures the FPGA.
7. **Programming:** Upload the bitstream to the FPGA.

8. What parameters of the module being developed are important to obtain during its synthesis, placement, and routing in FPGA CAD?

When performing synthesis, placement, and routing in FPGA CAD, the following parameters are important:

1. **Timing Constraints:** Clock period, setup/hold time constraints, and path delays.
2. **Resource Utilization:** The number of logic elements (LUTs, flip-flops), memory blocks, and DSP blocks used by the module.
3. **Power Consumption:** Estimated power usage of the design.
4. **Critical Paths:** The longest delay path in the design that could limit the clock frequency.
5. **Placement and Routing Efficiency:** How well the logic is placed on the FPGA and how efficiently signals are routed between components.
6. **Fanout:** The number of components driven by a single signal (which affects the performance and timing).
7. **Area Usage:** The amount of physical FPGA resources used by the design.

These parameters ensure the design meets the required performance, area, and power consumption targets.